

Retail Loyalty & Rewards

Omni-channel retail customer loyalty and rewards system — OData V4 service, live dashboard, automated tests, and SAP BTP deployment documentation.

Project: retail-loyalty-rewards-capstone

Stack: Node.js 22 · @sap/cds 10 · OData V4 · SQLite / SAP HANA Cloud

Testing: 13 automated tests · browser smoke test

Date: August 2026

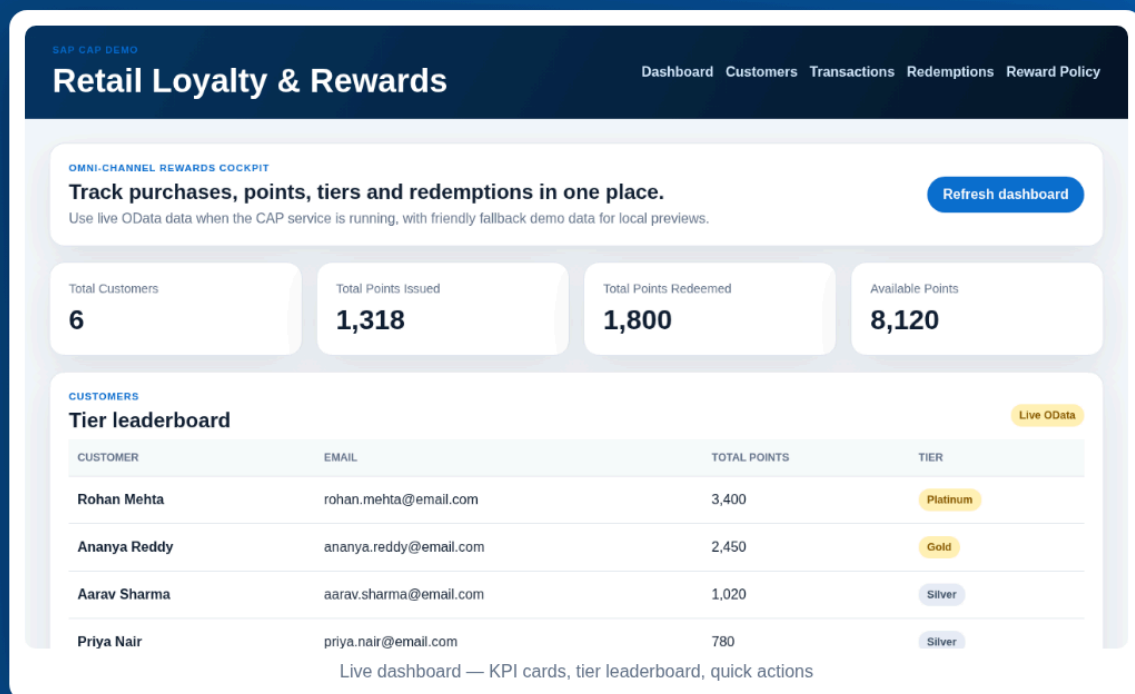


Table of Contents

-  **1. Project Overview**
-  **2. System Architecture**
-  **3. Data Model Design**
-  **4. CAP Service Definition**
-  **5. Business Logic**
-  **6. Agile Sprint Plan**
-  **7. SAPUI5/Fiori Application**
-  **8. Test Case Sheet**
-  **9. Deployment**
-  **10. Results & Screenshots**
-  **11. Challenges & Solutions**
-  **12. Learning Outcomes**
-  **13. Conclusion**

Retail Loyalty & Rewards — SAP CAP Project Documentation

A full-stack omni-channel retail customer loyalty and rewards system built on the SAP Cloud Application Programming Model (CAP), Node.js.

1. Project Overview

1.1 Project Title

Retail Loyalty & Rewards — Omni-Channel Loyalty Management on SAP CAP

1.2 Business Scenario

A retail chain sells through two channels — **Online** and **Store**. Every purchase earns loyalty points, and points can be redeemed against vouchers and discounts. Customers move up tiers (Bronze → Silver → Gold → Platinum) as their balance grows, which drives repeat business.

1.3 Problem Statement

Manual, channel-siloed loyalty tracking is error-prone: points miscalculated per channel, balances drifting out of sync, and no single dashboard for staff. The business needs one system that:

- computes points consistently per channel,
- keeps balances and tiers consistent in real time,
- blocks over-redemption,
- gives staff a single live cockpit.

1.4 Objective

Deliver a CAP-based loyalty service exposing all entities over **OData V4** with two custom actions (`recordPurchase`, `redeemPoints`), a live dashboard with KPI cards and quick actions, automated tests, and seed data — deployable to SAP BTP with SAP HANA Cloud.

1.5 Scope

In scope: Customers, Transactions, Redemptions, RewardPolicies; point computation (Online = 2 pts / ₹100, Store = 1 pt / ₹100); redemption validation; tier recalculation; dashboard UI; seed data; automated tests; deployment guides.

Out of scope: Authentication/authorization, multi-tenant isolation, refunds/reversals, real-time notifications.

1.6 User Roles

- **Customer** — earns and spends points via purchases and redemptions.
- **Store staff / cashier** — records purchases and redemptions at the till.

- **Loyalty admin** — monitors KPIs, customer tiers, and reward policies.

1.7 Key Features

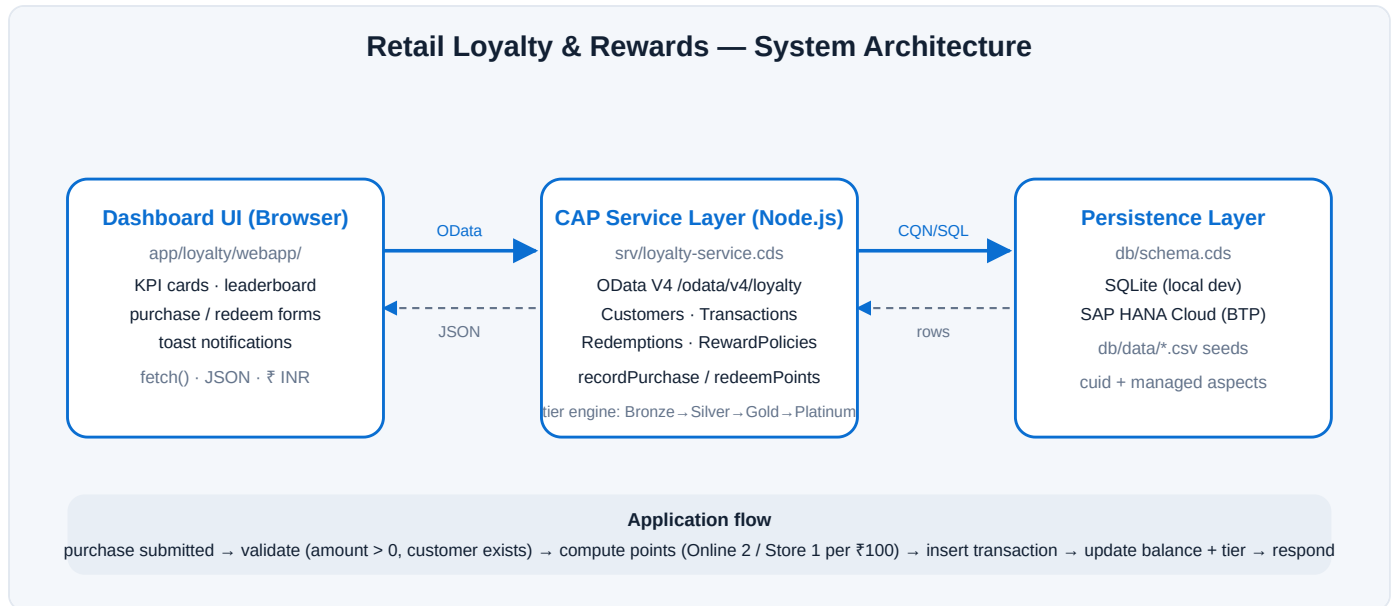
- OData V4 CRUD for 4 entities + 2 custom actions
- Channel-aware point engine with floor rounding
- Insufficient-balance and invalid-input validation with user-friendly errors
- Automatic tier recalculation after every purchase and redemption
- Live dashboard with KPI cards, tier leaderboard, activity feeds, quick actions, toast feedback
- Demo-mode fallback when the service is offline
- Seed data (6 customers, 6 transactions, 3 redemptions, 3 policies)
- 13 automated tests + browser smoke test

1.8 Technology Stack

Layer	Technology
Backend	Node.js 22, @sap/cds 10, CDS (Core Data Services)
Protocol	OData V4
Database	SQLite (local) / SAP HANA Cloud (BTP production)
Frontend	Vanilla HTML/CSS/JS (no build step)
Testing	@cap-js/cds-test, chai, headless Chrome smoke test
Deployment	SAP BTP Cloud Foundry, SAP HANA HDI, Business Application Studio

2. System Architecture

2.1 Architecture Diagram



2.2 Application Flow

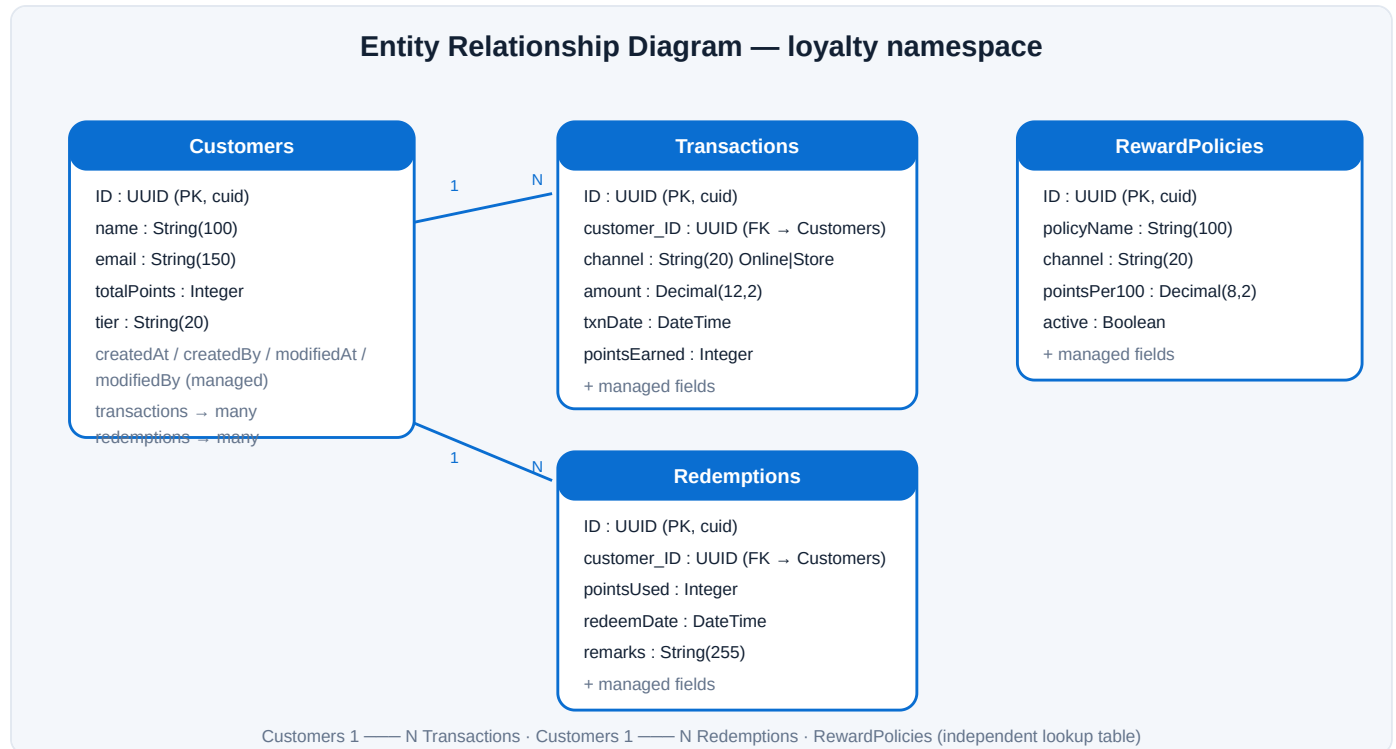
1. Staff opens the dashboard; it fetches all four entity sets from `/odata/v4/loyalty/`.
2. Recording a purchase POSTs `recordPurchase {customerID, channel, amount}`.
3. The handler validates, computes points ($\text{amount} / 100 \times \text{rate}$), inserts the transaction, updates the customer's `totalPoints`, and recomputes `tier`.
4. Redeeming POSTs `redeemPoints {customerID, pointsUsed, remarks}`; the handler rejects over-redemption, deducts the balance, and recomputes the tier.
5. The dashboard re-reads the entities and updates KPIs, lists, and toasts.

2.3 Component Description

- `db/schema.cds` — domain model: Customers, Transactions, Redemptions, RewardPolicies (all `cuid` + `managed`).
- `srv/loyalty-service.cds` — service definition: projections + actions, OData V4 at `/odata/v4/loyalty/`.
- `srv/loyalty-service.js` — action implementations and the tier engine.
- `app/loyalty/webapp/` — static dashboard served by CAP.
- `db/data/*.csv` — seed data auto-loaded on deploy.
- `test/` — automated API tests + browser smoke script.

3. Data Model Design

3.1 Entity Relationship Diagram



3.2 Customer Entity

Customers : cuid, managed — name, email, totalPoints (default 0), tier (default 'Bronze'), with transactions and redemptions backlink associations.

3.3 Transaction Entity

Transactions : cuid, managed — customer association (FK customer_ID), channel ('Online'|'Store'), amount Decimal(12,2), txnDate, pointsEarned.

3.4 Redemption Entity

Redemptions : cuid, managed — customer association, pointsUsed, redeemDate, remarks.

3.5 Relationships

- Customers **1** — **N** Transactions (a purchase belongs to exactly one customer)
- Customers **1** — **N** Redemptions
- RewardPolicies is an independent lookup table (channel → points-per-₹100, active flag)

4. CAP Service Definition

4.1 Service Architecture

`LoyaltyService` projects the four `loyalty` namespace entities and adds two unbound actions. Handlers are bound via the sibling-impl convention (`srv/loyalty-service.js` next to `srv/loyalty-service.cds`).

4.2 Exposed Entities

`Customers` , `Transactions` , `Redemptions` , `RewardPolicies` — all with CRUD over OData V4, including `customer_ID` foreign-key fields for the dashboard activity lists.

4.3 CRUD Operations

Standard OData V4 verbs: `GET /Customers` , `POST /Customers` , `GET /Customers(ID)` , `PATCH` , `DELETE` .
`$select` , `$orderby` , `$top` supported.

4.4 Custom Actions

- `recordPurchase(customerID : UUID, channel : String, amount : Decimal(12,2)) → Transactions`
- `redeemPoints(customerID : UUID, pointsUsed : Integer, remarks : String) → Redemptions`

4.5 API Endpoints

Endpoint	Method	Purpose
<code>/odata/v4/loyalty/Customers</code>	GET/POST	List / create customers
<code>/odata/v4/loyalty/Transactions</code>	GET	Purchase history
<code>/odata/v4/loyalty/Redemptions</code>	GET	Redemption history
<code>/odata/v4/loyalty/RewardPolicies</code>	GET	Active reward rules
<code>/odata/v4/loyalty/recordPurchase</code>	POST	Record purchase, earn points
<code>/odata/v4/loyalty/redeemPoints</code>	POST	Redeem points

5. Business Logic

5.1 Point Calculation

```
points = floor(amount / 100 × rate) .
```

5.2 Channel-Based Rewards

- Online:** 2 points per ₹100

- **Store:** 1 point per ₹100

5.3 Redemption Validation

- Customer must exist (404 otherwise)
- `pointsUsed` must be positive (400 otherwise)
- `pointsUsed` must not exceed `totalPoints` (400 "Insufficient loyalty points")

5.4 Point Deduction

On redemption, `totalPoints -= pointsUsed` and the tier is recomputed immediately — a big redemption can downgrade a customer.

5.5 Error Handling

Handlers return `req.error(status, message)` ; the dashboard renders `error.error.message` in a red toast.

Tier thresholds visualised:



Tiers: ≥ 3000 Platinum, ≥ 1500 Gold, ≥ 500 Silver, else Bronze.

6. Agile Sprint Plan

6.1 Sprint 1 — Entity model, mock data, service definitions

CDS schema, service projections, action signatures. (*commit: first commit*)

6.2 Sprint 2 — Point computation and redemption logic

Action handlers, validation, tier engine. (*commits: Details*)

6.3 Sprint 3 — Fiori dashboard

Static HTML/CSS/JS dashboard with KPIs, quick actions, demo-mode fallback. (*commits: Complete loyalty frontend dashboard*)

6.4 Sprint 4 — Deployment and testing

Seed data, 13 automated tests, browser smoke test, deployment guides, screenshots. (*this sprint*)

6.5 User Stories

- As staff, I record a purchase so the customer earns the right points for the channel.
- As staff, I redeem points so the balance drops by exactly the redeemed amount.
- As admin, I see KPIs and tiers so I can spot our best customers.

6.6 Acceptance Criteria

- TC-01..TC-06 pass in the automated suite (see §8)
- Dashboard shows "Live OData" against a running service
- Error toasts appear for invalid input and insufficient balance



7. SAPUI5/Fiori Application

7.1 Customer Dashboard

KPI cards (customers, points issued, redeemed, available), tier leaderboard table, recent transactions and redemptions feeds.

7.2 Staff Functionality

Quick-action forms for recording a purchase and redeeming points, with live validation and toast feedback.

7.3 Admin Dashboard

Reward-policy cards (Online 2/₹100, Store 1/₹100, tier refresh rule).

7.4 Screenshots

Desktop (1366px):

OMNI-CHANNEL REWARDS COCKPIT

Track purchases, points, tiers and redemptions in one place.

Use live OData data when the CAP service is running, with friendly fallback demo data for local previews.

Refresh dashboard

Total Customers

6

Total Points Issued

1,318

Total Points Redeemed

1,800

Available Points

8,120

CUSTOMERS

Tier leaderboard

Live OData

CUSTOMER	EMAIL	TOTAL POINTS	TIER
Rohan Mehta	rohan.mehta@email.com	3,400	Platinum
Ananya Reddy	ananya.reddy@email.com	2,450	Gold
Aarav Sharma	aarav.sharma@email.com	1,020	Silver
Priya Nair	priya.nair@email.com	780	Silver
Imran Khan	imran.khan@email.com	320	Bronze
Kavya Iyer	kavya.iyer@email.com	150	Bronze

Live service data loaded

Tablet (768px):

Retail Loyalty & Rewards

Dashboard Customers Transactions Redemptions Reward Policy

OMNI-CHANNEL REWARDS COCKPIT

Track purchases, points, tiers and redemptions in one place.

Use live OData data when the CAP service is running, with friendly fallback demo data for local previews.

Refresh dashboard

Total Customers

6

Total Points Issued

1,318

Total Points Redeemed

1,800

Available Points

8,120

CUSTOMERS

Tier leaderboard

Live OData

CUSTOMER	EMAIL	Live service data loaded	TOTAL POINTS	TIER
----------	-------	--------------------------	--------------	------

Mobile (390px):

Retail Loyalty & Rewards

Dashboard Customers Transactions

Redemptions Reward Policy

OMNI-CHANNEL REWARDS COCKPIT

Track purchases, points, tiers and redemptions in one place.

Use live OData data when the CAP service is running, with friendly fallback demo data for local previews.

Refresh dashboard

Total Customers

6

Total Points Issued

1,318

Total Points Redeemed

1,800

Live service data
loaded

Decision note: the dashboard deliberately stays plain HTML/CSS/JS rather than SAPUI5/ Fiori Elements. It has zero build step, renders instantly in BAS and BTP static hosting, and the CAP service already exposes full OData v4 so a Fiori Elements app can be added later without backend changes.

✓ 8. Test Case Sheet

8.1 Functional Tests

Customer creation (Bronze default), entity reads, activity feeds.

8.2 Business Logic Tests

Online/store point rates, floor rounding, accumulation across purchases.

8.3 Validation Tests

Invalid amount, missing channel, unknown customer, over-redemption, non-positive redemption.

8.4 Test Results

```
● ● ● npx cds test --profile local test/LoyaltyService.test.js

LoyaltyService OData APIs
  ✓ serves seeded Customers
  ✓ serves seeded Transactions, Redemptions and RewardPolicies

recordPurchase
  ✓ awards 2 points per ₹100 for Online purchases (TC-02)
  ✓ awards 1 point per ₹100 for Store purchases (TC-03)
  ✓ rounds points down and accumulates across purchases
  ✓ rejects invalid amount and missing channel
  ✓ rejects unknown customer (404)

redeemPoints
  ✓ deducts points within balance and recalculates tier (TC-04)
  ✓ rejects redeeming above balance (TC-05)
  ✓ rejects non-positive points
  ✓ rejects unknown customer (404)

tier recalculation (TC-06)
  ✓ applies Silver at 500, Gold at 1500, Platinum at 3000
  ✓ drops tier after redemption crosses below threshold

13 passed
1.426s
```

13/13 passing. Full mapping in `docs/TEST_CASE_SHEET.txt`.



9. Deployment

9.1 Prerequisites

Node.js 22+, npm, SAP Business Application Studio (for BTP), SAP HANA Cloud instance.

9.2 BTP Configuration

HANA HDI container service instance + Cloud Foundry org/space.

9.3 Build

`npm run build` (produces `gen/` artifacts).

9.4 Cloud Foundry Deployment

`cf push` or `mbt build && cf deploy` — full steps in `docs/DEPLOYMENT.txt`.

9.5 Deployment Commands

See `docs/DEPLOYMENT.txt` and `HANA_Deployment_Guide.md`.

9.6 Verification

```
cds watch --port 4044 . then curl OData

[cds] - successfully deployed to db.sqlite
[cds] - serving LoyaltyService { url: '/odata/v4/loyalty' }
[cds] - server listening on { url: 'http://localhost:4044' }

$ curl http://localhost:4044/odata/v4/loyalty/Customers
{
  "@odata.context": "$metadata#Customers",
  "value": [
    { "ID": "11111111-...", "name": "Aarav Sharma", "email": "aarav.sharma@email.com",
      "totalPoints": 1270, "tier": "Silver" },
    { "ID": "22222222-...", "name": "Ananya Reddy", "email": "ananya.reddy@email.com",
      "totalPoints": 2450, "tier": "Gold" },
    { "ID": "55555555-...", "name": "Rohan Mehta", "email": "rohan.mehta@email.com",
      "totalPoints": 3400, "tier": "Platinum" }
  ]
}

$ curl -X POST http://localhost:4044/odata/v4/loyalty/recordPurchase \
  -H "Content-Type: application/json" \
  -d '{"customerID":"11111111-...", "channel":"Online", "amount":1000}'
→ { "channel": "Online", "amount": "1000.00", "pointsEarned": 20 }

$ curl -X POST http://localhost:4044/odata/v4/loyalty/redeemPoints \
  -d '{"customerID":"11111111-...", "pointsUsed":99999, "remarks":""}'
→ 400 - Insufficient loyalty points
```



10. Results & Screenshots

Live-service verification (dashboard against `cds watch`):

OMNI-CHANNEL REWARDS COCKPIT

Track purchases, points, tiers and redemptions in one place.

Use live OData data when the CAP service is running, with friendly fallback demo data for local previews.

Refresh dashboard

Total Customers

6

Total Points Issued

1,318

Total Points Redeemed

1,800

Available Points

8,120

CUSTOMERS

Tier leaderboard

Live OData

CUSTOMER	EMAIL	TOTAL POINTS	TIER
Rohan Mehta	rohan.mehta@email.com	3,400	Platinum
Ananya Reddy	ananya.reddy@email.com	2,450	Gold
Aarav Sharma	aarav.sharma@email.com	1,020	Silver
Priya Nair	priya.nair@email.com	780	Silver
Imran Khan	imran.khan@email.com	320	Bronze
Kavya Iyer	kavya.iyer@email.com	150	Bronze

QUICK ACTION

Record purchase

Customer

Aarav Sharma · 1,020 pts

Channel

Online

Amount (₹)

0

Add points

Recent Transactions

Aarav Sharma +20 pts	Online · ₹1,000.00
Ananya Reddy +130 pts	Online · ₹6,500.00
Aarav Sharma +24 pts	Store · ₹2,400.00
Priya Nair +22 pts	Online · ₹1,100.00
Rohan Mehta +1,000 pts	Online · ₹50,000.00

Recent Redemptions

Aarav Sharma -250 pts	Voucher redemption
Ananya Reddy -800 pts	Festival voucher
Aarav Sharma -250 pts	Checkout discount
Rohan Mehta -500 pts	Gift card

QUICK ACTION

Redeem points

Customer

Aarav Sharma · 1,020 pts

Points

99999

Remarks

Voucher redemption

Redeem

Business rules

ONLINE

Online purchase

2.00 points per ₹100

STORE

Store purchase

1.00 points per ₹100

ALL

Tier refresh

Recalculate tier after every change

- Seeded data loads immediately after startup (6 customers, 4 tiers represented).
- `recordPurchase` live: Aarav Sharma ₹1,000 Online → +20 pts, balance 1,250 → 1,270, KPI "Points Issued" 1,298 → 1,318, new activity row.
- `redeemPoints` live: 250-pt redemption → balance 1,270 → 1,020, KPI "Points Redeemed" 1,550 → 1,800, new redemption row.
- Error paths live: ₹0 purchase → "Valid customer, channel and amount are required"; 99,999-pt redemption → "Insufficient loyalty points" (both as red toasts).
- Browser smoke test (`npm run test:smoke`) — 11/11 assertions pass.

11. Challenges & Solutions

Challenge	Solution
Tests hit <code>no such table</code> in-memory	Removed explicit sqlite file credentials from default config so <code>cds test --in-memory?</code> bootstraps a clean in-memory DB; <code>deploy:local</code> still writes the file DB explicitly
<code>test:unit</code> script pointed at a non-existent file	Corrected to <code>test/LoyaltyService.test.js</code>
Managed CSV headers were incomplete	Regenerated CSVs with minimal columns (ID, business fields, FKs); CAP fills managed columns on deploy
Tier math bug in a test (₹75,000 online ≠ 3,000 pts)	Fixed test input to ₹150,000 → exactly 3,000 pts (2 pts/₹100)
Background shells resolve system node v20	Exported absolute nvm node v22 PATH in server and smoke-script launches

12. Learning Outcomes

- CAP action handlers, `req.error`, and sibling-impl discovery
- CDS associations → `customer_ID` FK exposure in OData projections
- CSV seed data mechanics and minimal-column seeding
- `cds test` in-memory bootstrap behavior
- Building a zero-build static UI against OData v4 with graceful fallback

13. Conclusion

The project delivers a complete, tested, and verified omni-channel loyalty system: consistent channel-based point earning, guarded redemption, automatic tiering, a live dashboard, realistic seed data, 13 automated tests, and a browser smoke gate — packaged with full deployment documentation for SAP BTP and HANA Cloud.